



UNIVERSIDAD ANDRÉS BELLO
FACULTAD DE INGENIERÍA
INGENIERÍA CIVIL INFORMÁTICA

Sistema de Gestión Inteligente de Recetas y Presupuestos para Repostería y Cocina

Informe de Avance – Proyecto de Título

Infraestructura, Base de Datos, Backend, Frontend y Prototipo IA

BRUNO IGNACIO PAINAMIL NAVIA
Profesor guía: Pablo Hernán Schwarzenberg Riveros

SANTIAGO - CHILE
Junio, 2026

Índice General

Abstract.....	4
Resumen.....	4
1. Introducción	5
2. Base de Datos	6
2.1 tecnología utilizada	6
2.2 Diagrama de Entidades	6
2.3 Descripción de las tablas.....	8
users — Usuarios.....	8
ingredients — Catalogo de ingredientes.....	8
recipes y recipe_ingredients — Recetas.....	8
substitutes — Sustitutos de ingredientes (Modulo IA)	9
quotes y quote_items — Cotizaciones	9
unit_conversions — Conversiones de unidades	9
3. Backend - API REST.....	10
3.1 Lenguaje y Frameworks utilizados	10
3.2 Estructura del BackEnd	10
3.3 Endpoints de la API	10
Autenticacion.....	10
Ingredientes	10
Recetas.....	11
Cotizaciones	11
3.4 Formula de cálculo de cotizaciones (temporal)	11
3.5 Testeo de la API	12
4. FrontEnd - Aplicación Web.....	14
4.1 Tecnología utilizada	14
4.2 Conexión con Backend.....	14
4.3 Módulos	14
Inicio de sesión y registro	14
Módulo de Recetas	16
Módulo de Ingredientes.....	18
Módulo de Cotizaciones	19
4.4 Diseño visual.....	20
5. Infraestructura y Despliegue	21
5.1 Docker y Docker Compose	21
5.2 Variables de entorno	21
6. Módulo de Inteligencia Artificial - Prototipo	22
6.1 Contexto y objetivo.....	22
6.2 Síntesis de Experimentación con Algoritmos de Aprendizaje Automático	22

6.3 Datasets utilizados	23
6.4 Método de aprendizaje: K-Nearest Neighbors (KNN).....	24
6.5 Métricas del prototipo.....	24
6.6 Estado actual y limitaciones	25
6.7 Trabajo pendiente para el módulo IA.....	25
7. Plan de Trabajo	27
8. Referencias.....	28

Abstract

This report presents the first functional increment of the Intelligent Recipe and Budget Management System for Bakery and Cooking. The system is aimed at small businesses and independent workers in the gastronomy sector who face difficulties in organizing recipes, calculating ingredient quantities, and preparing quotations for large orders. This increment covers the design and deployment of the complete technological infrastructure: relational database, REST API backend, and web frontend. Additionally, a prototype of the artificial intelligence module for ingredient substitution recommendation is documented, based on the K-Nearest Neighbors algorithm applied to culinary property vectors.

Resumen

El presente informe documenta el avance funcional del Sistema de gestión Inteligente de recetas y presupuestos para repostería y Cocina. El sistema está orientado a pequeños negocios y emprendedores del rubro gastronómico que enfrentan dificultades para organizar recetas, calcular cantidades de ingredientes y elaborar presupuestos o cotizaciones para pedidos de gran volumen. Este incremento cubre el diseño y despliegue de la infraestructura tecnológica completa: base de datos relacional, BackEnd REST API y FrontEnd web. Además, se documenta un prototipo del módulo de Machine Learning para la recomendación de sustitutos de ingredientes, basado en el algoritmo K-Nearest Neighbors aplicado sobre vectores de propiedades culinarias.

1. Introducción

La repostería y la cocina artesanal son actividades que exigen precisión técnica, especialmente en el contexto de pequeños negocios que deben gestionar simultáneamente el control de recetas, el cálculo de costos y la generación de presupuestos para clientes. Existe una brecha tecnológica clara entre las aplicaciones de recetas orientadas al hogar y los sistemas ERP corporativos diseñados para grandes cadenas. Ninguno de estos extremos sirve de manera adecuada al segmento de pymes y emprendedores gastronómicos.

Este informe presenta el incremento funcional del sistema propuesto, desarrollado bajo la metodología Scrum adaptada al trabajo individual. El resultado de este incremento es una aplicación web completamente operativa que permite a un usuario autenticado gestionar sus recetas, escalar ingredientes automáticamente según el número de porciones, generar cotizaciones con cálculo de márgenes, y visualizar un prototipo del módulo de recomendación de sustitutos basado en inteligencia artificial.

La arquitectura del sistema sigue el patrón de tres capas: una base de datos PostgreSQL como capa de persistencia, una API REST construida con Python y FastAPI como capa de lógica de negocio, y una aplicación de página única (SPA) construida con React como capa de presentación. Todo el sistema se levanta con un único comando mediante Docker Compose, lo que garantiza la replicabilidad del entorno en cualquier máquina.

2. Base de Datos

2.1 tecnología utilizada

La base de datos fue implementada con PostgreSQL, un sistema gestor de bases de datos relacional de Código abierto, ampliamente utilizado en la industria por su robustez, soporte de transacciones y compatibilidad con lenguajes de programación modernos. PostgreSQL fue elegido por sobre otras alternativas como MySQL por su soporte nativo del tipo de dato UUID como clave primaria, lo que garantiza la unión de registros incluso en entornos distribuidos.

El esquema de la base de datos fue escrito en SQL estándar y se carga automáticamente al inicializar el contenedor de base de datos por primera vez. Además, incluye datos de prueba precargados para que la aplicación funcione desde el primer arranque.

2.2 Diagrama de Entidades

El siguiente diagrama muestra las 8 tablas del sistema y sus relaciones. Las flechas indican que una tabla referencia a otra mediante una clave foránea (FK).

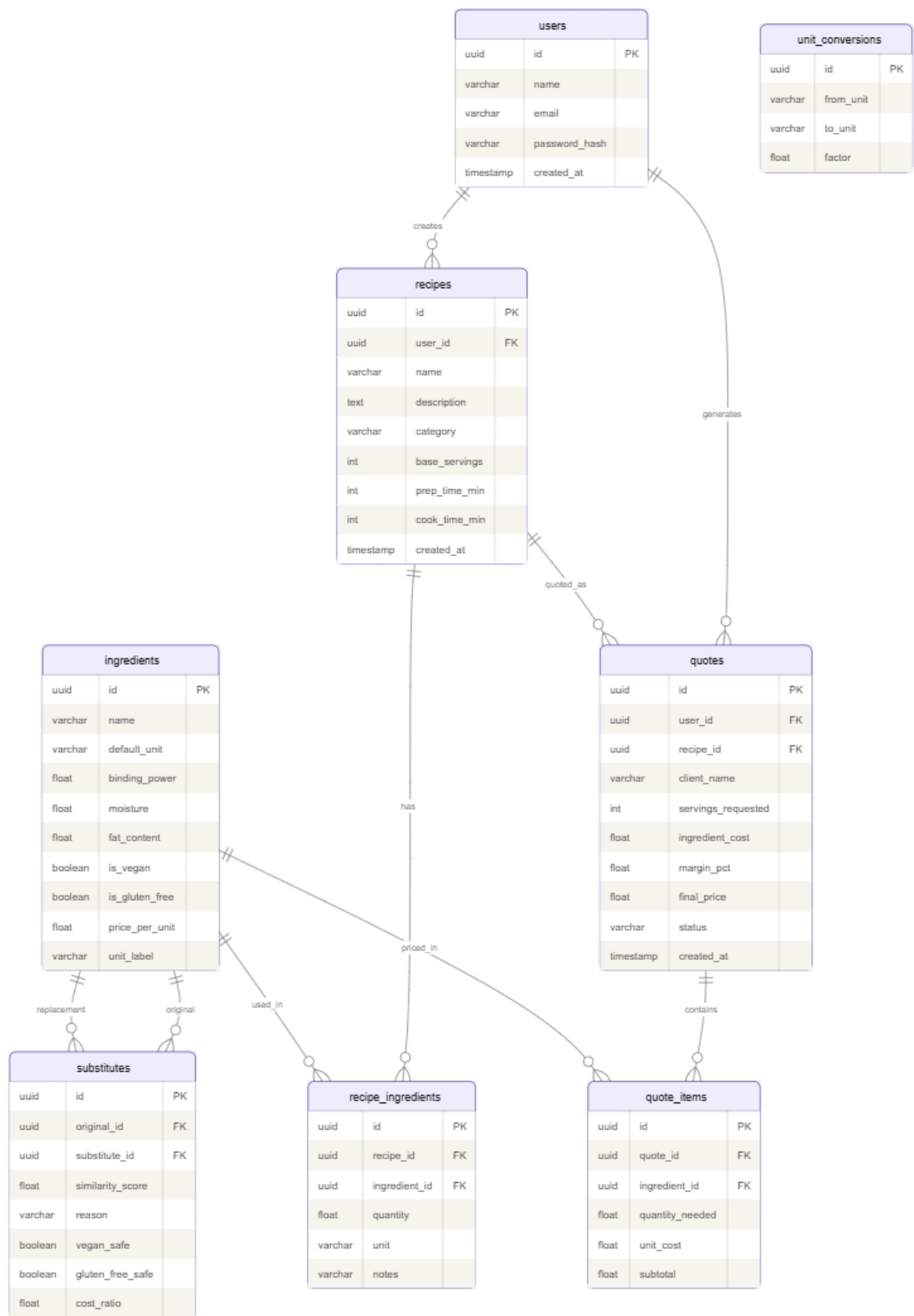


Figura 1: Diagrama Entidad-relación del sistema. Las 8 tablas cubren los módulos de usuarios, recetas, ingredientes, cotizaciones y el módulo IA.

2.3 Descripción de las tablas

A continuación, se describe el propósito de cada tabla del sistema:

users — Usuarios

Almacena los datos de los usuarios registrados en el sistema (nombre, email y contraseña). La contraseña nunca se guarda en texto plano: se aplica un algoritmo de hash llamado “bcrypt” que la convierte en una cadena irreversible. Cada usuario solo puede ver y editar sus propias recetas y cotizaciones.

ingredients — Catalogo de ingredientes

Es una de las tablas más importantes del sistema. Además de los datos básicos de cada ingrediente (nombre, unidad de medida y precio por unidad), almacena cinco propiedades culinarias en escala del 0 al 10 que son utilizadas por el modelo de inteligencia artificial para calcular similitudes entre ingredientes:

- `binding_power`: que tan bien une o da estructura a la mezcla (por ejemplo, el huevo tiene valor 9).
- `moisture`: que tanta humedad aporta al producto final (la leche tiene valor 9).
- `fat_content`: su contenido de grasa (la mantequilla tiene valor 10).
- `sweetness`: su nivel de dulzor (el azúcar tiene valor 10).
- `leavening`: su capacidad de hacer crecer la masa (el polvo de hornear tiene valor 10).

Respecto a los precios, en esta versión de prueba los valores están definidos manualmente en pesos chilenos (CLP) en base a precios de referencia de mercado. Sin embargo, una mejora futura importante es permitir que el propio usuario edite los precios de sus ingredientes directamente en la aplicación, ya que los costos reales varían según el proveedor y la región. también se contempla a futuro la integración con fuentes de precios en tiempo real para que los presupuestos sean aún más precisos.

recipes y recipe_ingredients — Recetas

La tabla `recipes` guarda los datos generales de cada receta: nombre, categoría (torta, galleta, pan, etc.), número de porciones base, tiempos de preparación e instrucciones. La tabla `recipe_ingredients` actúa como puente entre una receta y sus ingredientes, guardando la cantidad y la unidad específica de cada ingrediente en esa receta. Esta estructura es la que permite el escalado automático de porciones: si una receta base es para 12 personas y se necesita para 36, el sistema simplemente multiplica cada cantidad por 3.

substitutes — Sustitutos de ingredientes (Modulo IA)

Esta tabla guarda los pares de sustitutos calculados por el modelo de inteligencia artificial. Por ejemplo: "el gel de linaza puede reemplazar al huevo con un 85% de similitud". Incluye el puntaje de similitud, la razón textual explicando porque son similares, y si el sustituto es apto para dietas veganas o sin gluten. Esta tabla es llenada desde el notebook de entrenamiento en Google Colab y luego importada a la base de datos de producción.

quotes y quote_items — Cotizaciones

La tabla quotes guarda los datos de cada cotización generada: la receta base, el cliente, las porciones solicitadas y los porcentajes de cálculo. La tabla quote_items guarda el detalle de cada ingrediente escalado con su costo unitario y subtotal. Una cotización puede tener cuatro estados: borrador, enviada, aceptada o rechazada, lo que permite hacer seguimiento de los pedidos.

unit_conversions — Conversiones de unidades

Tabla auxiliar que permite al sistema convertir automáticamente entre distintas unidades de medida comunes en repostería: tazas, gramos, mililitros, cucharadas y cucharaditas. Viene precargada con 13 conversiones estándar.

3. Backend - API REST

3.1 Lenguaje y Frameworks utilizados

El BackEnd fue desarrollado en Python, lenguaje elegido principalmente por dos razones: es el lenguaje dominante en el ecosistema de inteligencia artificial y Machine Learning (scikit-learn, numpy, pandas), lo que permite que el mismo lenguaje utilizado para entrenar el modelo KNN sea también el que construye la API. Esto evita la necesidad de puentes o conversiones entre tecnologías.

Como Framework web se utilizó FastAPI, que permite construir una API REST de forma rápida y con validación automática de los datos. Una ventaja clave de FastAPI es que genera automáticamente una documentación interactiva disponible en el navegador (Swagger UI), lo que facilita el testeo de cada Endpoints sin necesidad de herramientas adicionales.

3.2 Estructura del BackEnd

El Código del BackEnd está organizado en cuatro capas con responsabilidades separadas:

- Modelos: definen la estructura de las tablas de la base de datos en Python, usando el ORM SQLAlchemy.
- Esquemas: definen el formato esperado de los datos que llegan y salen de la API, usando Pydantic para validación automática.
- Servicios: contienen la lógica de negocio, como el cálculo de escalado de ingredientes o la generación del precio final de una cotización.
- Routers: definen los Endpoints HTTP y delegan el trabajo a los servicios.

3.3 Endpoints de la API

A continuación, se describen los principales grupos de Endpoints disponibles en el sistema:

Autenticacion

Método	Ruta	Función
POST	/auth/register	Registrar un nuevo usuario con email y contraseña
POST	/auth/login	Iniciar sesión y obtener token de acceso (JWT)
GET	/auth/me	Ver los datos del usuario que está conectado

Ingredientes

Método	Ruta	Función
GET	/ingredients	Listar todos los ingredientes, con filtros de búsqueda

Método	Ruta	Función
POST	/ingredients	Agregar un nuevo ingrediente al catalogo
PUT	/ingredients/{id}	Editar los datos de un ingrediente existente
GET	/ingredients/{id}/substitutes	Ver los sustitutos recomendados por el modelo IA

Recetas

Metodo	Ruta	Función
GET	/recipes	Listar las recetas del usuario con buscador
POST	/recipes	Crear una nueva receta con sus ingredientes
PUT	/recipes/{id}	Editar una receta existente
DELETE	/recipes/{id}	Eliminar una receta
GET	/recipes/{id}/scale?servings=N	Escalar los ingredientes a N porciones con costo estimado

Cotizaciones

Metodo	Ruta	Funcion
POST	/quotes	Generar una cotización automática desde una receta
GET	/quotes	Ver todas las cotizaciones del usuario
GET	/quotes/{id}	Ver el detalle completo de una cotización
PATCH	/quotes/{id}/status	Cambiar el estado de la cotización
DELETE	/quotes/{id}	Eliminar una cotización

3.4 Formula de cálculo de cotizaciones (temporal)

Al generar una cotización, el sistema sigue tres pasos automáticos:

Primero, escala todos los ingredientes de la receta al número de porciones solicitadas por el cliente, multiplicando cada cantidad por el factor de escala correspondiente.

Segundo, calcula el costo total de ingredientes sumando el costo de cada uno (cantidad escalada multiplicada por el precio por unidad).

Tercero, aplica la siguiente fórmula para calcular el precio final sugerido al cliente:

$$\text{Precio Final} = \text{Costo Ingredientes} \times (1 + \text{Gastos Indirectos}\%) \times (1 + \text{Margen}\%)$$

Los gastos indirectos cubren costos como electricidad, gas y tiempo de preparación, que el usuario define como porcentaje.

El margen es la ganancia deseada sobre el costo total. Por ejemplo, con un costo de ingredientes de \$5.000, un 20% de gastos indirectos y un 30% de margen, el precio sugerido sería $\$5.000 \times 1.20 \times 1.30 = \7.800 .

3.5 Testeo de la API

FastAPI genera automáticamente una interfaz Swagger UI disponible en <http://localhost:8000/docs> (testeo local en la máquina) donde es posible probar cada Endpoint de forma interactiva directamente desde el navegador, incluyendo autenticación con usuario y contraseña.



Figura 2: Documentación interactiva Swagger UI con todos los Endpoints disponibles del sistema.

Available authorizations

Scopes are used to grant an application different levels of access to data on behalf of the end user. Each API may declare one or more scopes.
API requires the following scopes. Select which ones you want to grant to Swagger UI.

OAuth2PasswordBearer (OAuth2, password)

Authorized

Token URL: /auth/login
Flow: password
username: bruno@tesis.cl
password: *****
Client credentials location: basic
client_secret: *****

Logout

Close

Figura 3: Panel de autorización OAuth2 en Swagger UI. El usuario ingresa sus credenciales y el sistema emite un token JWT para las solicitudes protegidas.

Available authorizations

Scopes are used to grant an application different levels of access to data on behalf of the end user. Each API may declare one or more scopes.
API requires the following scopes. Select which ones you want to grant to Swagger UI.

OAuth2PasswordBearer (OAuth2, password)

Token URL: /auth/login
Flow: password
username:
password:
Client credentials location: Authorization header
client_id:
client_secret:

Authorize

Close

Figura 4: Respuesta exitosa del Endpoints de registro de usuario, mostrando el objeto creado con su identificador único.

4. FrontEnd - Aplicación Web

4.1 Tecnología utilizada

La interfaz de usuario fue desarrollada con React 18, una librería de JavaScript ampliamente utilizada para construir aplicaciones web interactivas. Como herramienta de construcción y servidor de desarrollo se utilizó Vite, que ofrece tiempos de arranque muy rápidos y actualización en tiempo real al editar el código.

La aplicación sigue el patrón SPA (Single Page Application): toda la interfaz se carga una sola vez en el navegador y las navegaciones entre secciones no requieren recargar la página, lo que entrega una experiencia fluida similar a una aplicación de escritorio.

4.2 Conexión con Backend

La comunicación entre el FrontEnd y el Backend se realiza mediante solicitudes HTTP a la API REST, usando la librería “Axios”. Cuando el usuario inicia sesión, el sistema recibe un token JWT del Backend y lo guarda en el navegador. A partir de ese momento, cada solicitud que hace el FrontEnd incluye automáticamente ese token en el encabezado, de modo que el Backend puede verificar la identidad del usuario sin que este tenga que ingresar sus credenciales nuevamente.

Para el manejo del estado del servidor (listas de recetas, ingredientes, cotizaciones) se utiliza React Query, que gestiona automáticamente la carga de datos, el caché y la actualización de la interfaz cuando se detectan cambios.

4.3 Módulos

Inicio de sesión y registro

La pantalla de autenticación permite tanto ingresar con una cuenta existente como crear una cuenta nueva. El formulario alterna entre ambos modos sin cambiar de página. Una vez autenticado, el usuario es redirigido automáticamente a su lista de recetas.

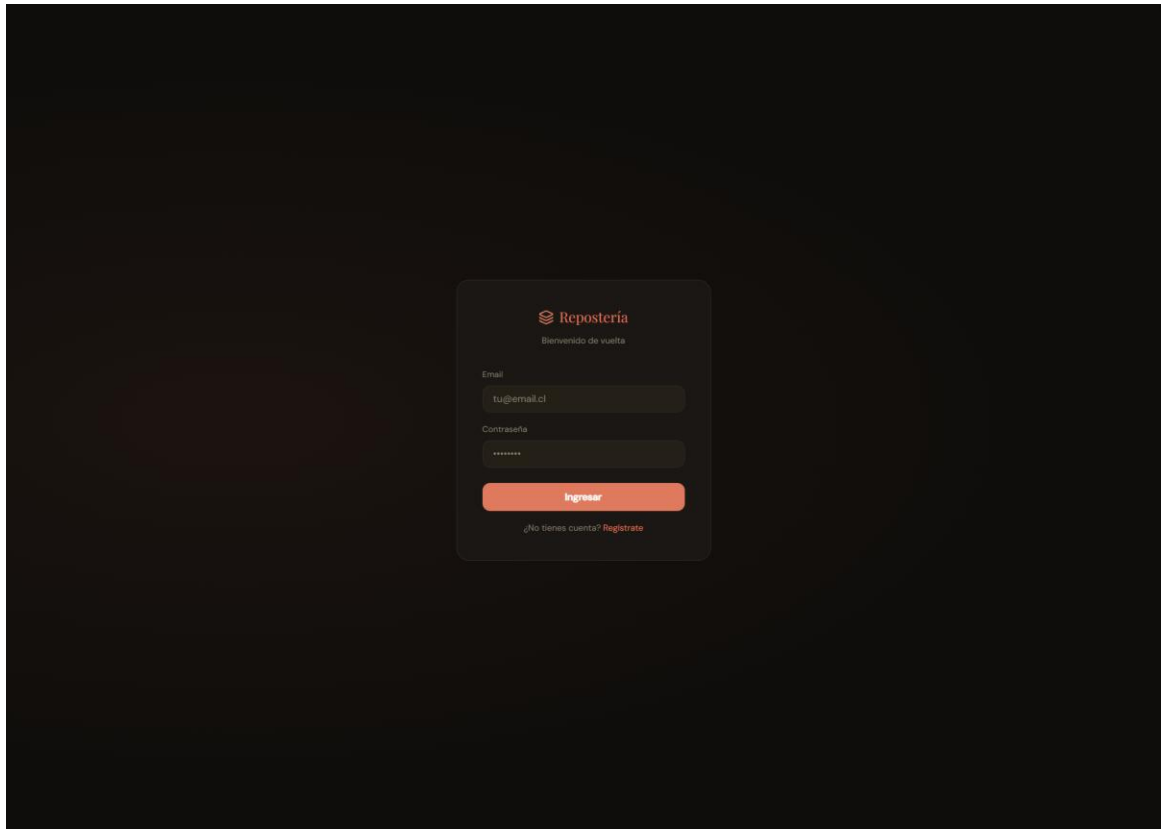


Figura 5: Interfaz de la aplicación web mostrando el inicio de sesión con cuenta ya registrada.

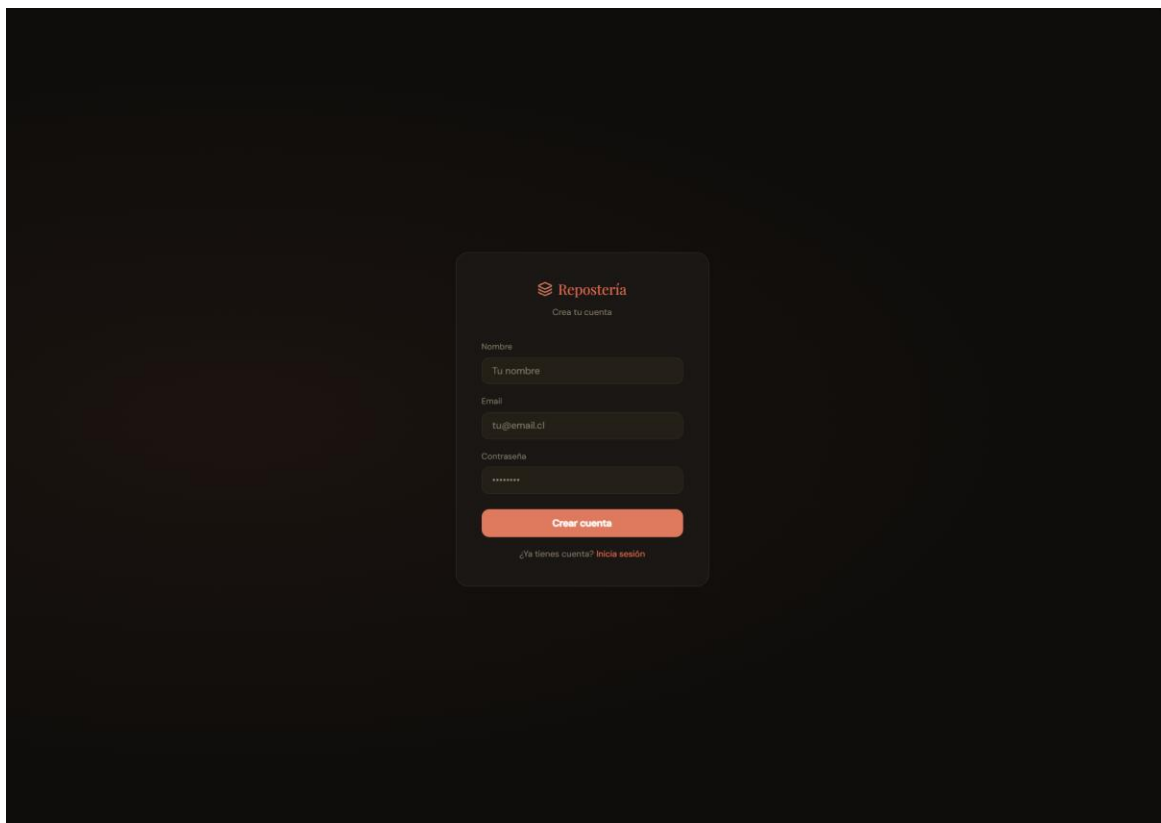


Figura 6: Interfaz de la aplicación web mostrando el formulario de registro.

Módulo de Recetas

Permite crear recetas con nombre, categoría, descripción, tiempos de preparación y lista de ingredientes. Cada ingrediente se selecciona del catálogo y se especifica su cantidad y unidad para esa receta en concreto. Las recetas existentes pueden editarse o eliminarse. El botón "Escalar" abre un panel donde el usuario ingresa el número de porciones deseadas y el sistema muestra en tiempo real las cantidades ajustadas de cada ingrediente junto al costo estimado de materias primas.

The screenshot shows the 'Nueva receta' (New recipe) form in the Repostería application. The form is a modal window with a dark background and orange accents. It contains the following fields and controls:

- Nombre ***: A text input field with the placeholder 'Ej. Tarta de chocolate'.
- Categoría**: A dropdown menu with the selected option 'Sin categoría'.
- Descripción**: A text area with the placeholder 'Descripción breve de la receta...'.
- Porciones base**: A numeric input field with the value '8'.
- Prep. (min)**: A numeric input field with the value '0'.
- Cocción (min)**: A numeric input field with the value '0'.
- Ingredientes**: A section with a '+ Agregar' button.
- Instrucciones**: A text area with the placeholder 'Paso 1: Precalentar el horno...'.
- Buttons**: 'Cancelar' and 'Guardar receta' buttons at the bottom right.

The background shows a sidebar with 'Recetas', 'Ingredientes', and 'Cotizaciones' options, and a main area with 'Mis Recetas' and a search bar.

Figura 7: Interfaz de la aplicación web mostrando el formulario de añadir una nueva receta.

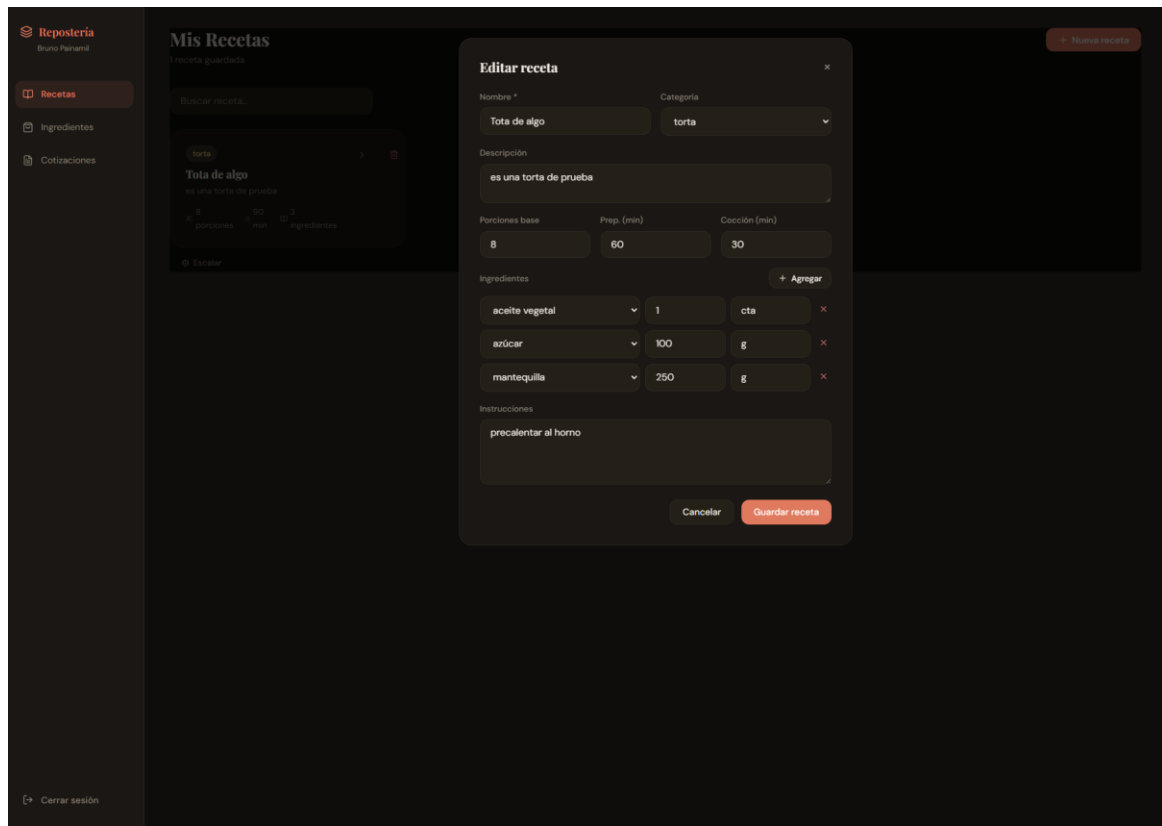


Figura 8: Interfaz de la aplicación web mostrando el formulario de editar una receta ya existente.

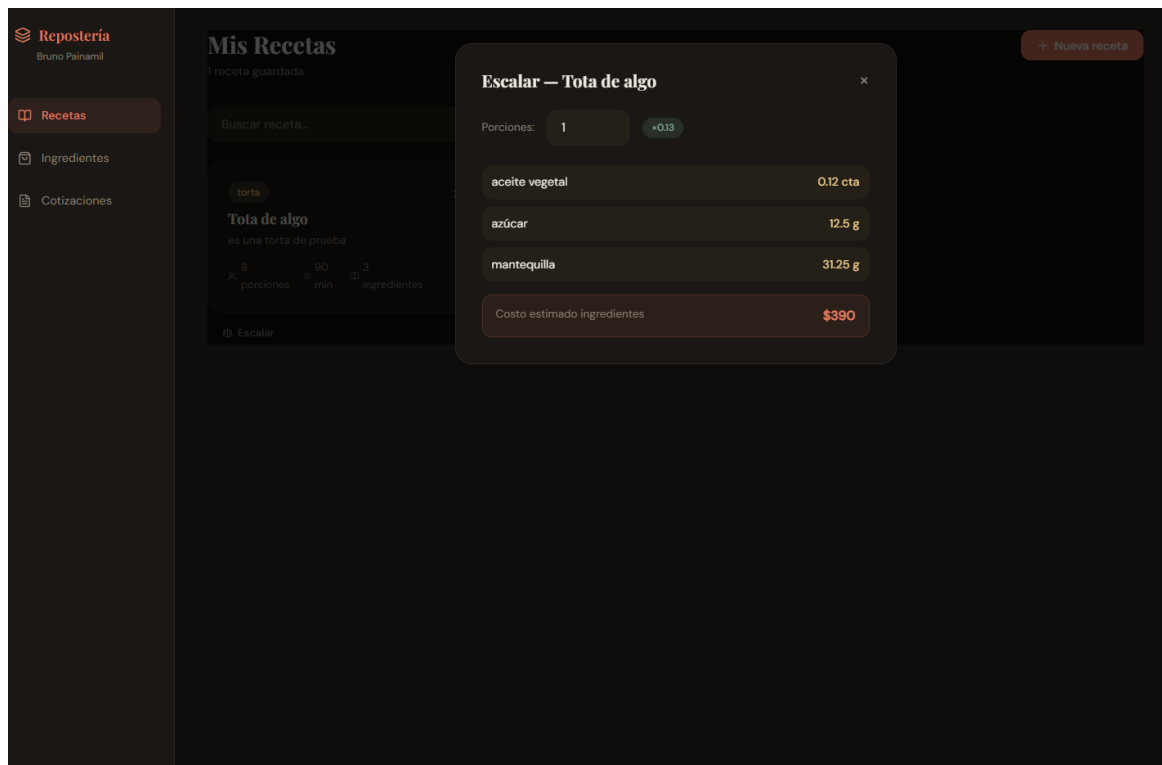


Figura 9: Interfaz de la aplicación web mostrando el apartado de escalado para una porción indicando medidas y el precio.

Módulo de Ingredientes

Muestra el catálogo completo de ingredientes disponibles en el sistema. Cada ingrediente presenta barras visuales que representan sus cinco propiedades culinarias (aglutinante, humedad, grasa, dulzor, leudante). Al hacer clic en un ingrediente, el panel lateral muestra los sustitutos recomendados por el modelo de IA, con su porcentaje de similitud, la razón del porque pueden reemplazarse y si son aptos para dietas veganas o sin gluten.

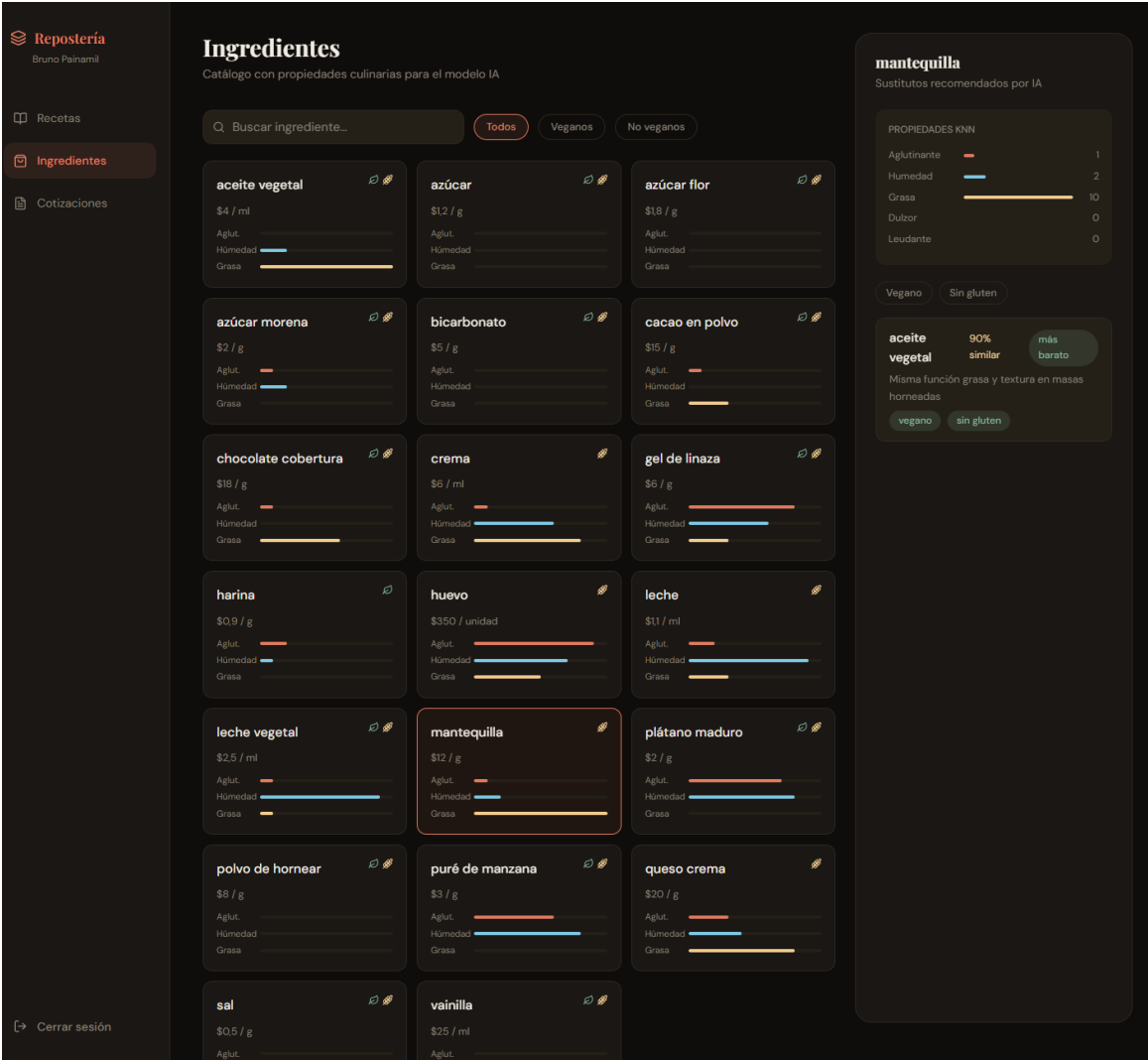


Figura 10: Interfaz de la aplicación web mostrando la página de ingredientes, en la imagen se aprecia el ingrediente mantequilla seleccionado. A la derecha se indica el detalle del ingrediente y su ingrediente equivalente químicamente con opción vegano y sin gluten.

Este módulo se encuentra en un periodo de prueba temporal debido a la cantidad reducida de ingredientes en la base de datos y está en proceso de mejora del modelo de sustitución por Machine Learning.

Módulo de Cotizaciones

El usuario selecciona una receta, especifica las porciones que pide el cliente, define el porcentaje de gastos indirectos y el margen de ganancia deseado. Con esos datos, el sistema genera automáticamente la cotización con el desglose de ingredientes escalados y el precio final sugerido.

Cada cotización puede cambiar de estado (borrador, enviada, aceptada o rechazada) para hacer seguimiento del pedido. Un panel de resumen muestra el total de ingresos acumulados de cotizaciones aceptadas.

Figura 11: Interfaz de la aplicación web mostrando el formulario de creación de cotización con selección de la receta y cantidades



Figura 12: Interfaz de la aplicación web mostrando una cotización realizada con una receta ya existente con su estado como borrador.

4.4 Diseño visual

La aplicación utiliza un diseño propio de tema oscuro con colores cálidos que evocan el ambiente de una repostería artesanal. Se utilizan dos tipografías: Playfair Display para los títulos, que aporta elegancia editorial, y DM Sans para el texto funcional, que garantiza legibilidad. El sistema de colores incluye un color terracota como color de acción principal y un dorado para indicadores secundarios.

5. Infraestructura y Despliegue

5.1 Docker y Docker Compose

Todo el sistema se ejecuta mediante Docker, una herramienta que permite empaquetar aplicaciones en contenedores aislados que funcionan de la misma forma en cualquier computador, independientemente del sistema operativo. El proyecto utiliza Docker Compose para orquestar dos contenedores que trabajan en conjunto:

- Contenedor de base de datos (reposteria-db): ejecuta PostgreSQL con un volumen persistente, lo que significa que los datos no se pierden cuando el sistema se apaga.
- Contenedor del Backend (reposteria-api): ejecuta Python con FastAPI. Este contenedor espera a que la base de datos esté lista antes de arrancar, evitando errores de conexión.

Para levantar el sistema completo desde cero solo se necesita ejecutar un único comando en la terminal: **“docker-compose up –build”**

El FrontEnd de React se ejecuta por separado con “npm run dev” durante el desarrollo.

5.2 Variables de entorno

Los datos sensibles como la clave secreta para los tokens JWT y la contraseña de la base de datos se configuran mediante variables de entorno definidas en un archivo “.env”, que no se sube al repositorio de Código. Esto es una práctica estándar de seguridad para evitar exponer credenciales.

6. Módulo de Inteligencia Artificial - Prototipo

6.1 Contexto y objetivo

Uno de los aspectos diferenciadores de este sistema es la capacidad de recomendar sustitutos de ingredientes de forma inteligente. Esto es especialmente útil para reposteros que necesitan adaptar una receta porque un ingrediente no está disponible, es muy caro, o el cliente tiene restricciones alimentarias (vegano, sin gluten).

Para desarrollar y probar este módulo se utilizó Python en Google Colab, un entorno de programación gratuito basado en navegador que permite ejecutar Código Python con facilidad. Google Colab fue elegido porque facilita el trabajo con librerías de machine Learning sin necesidad de configurar un entorno local.

6.2 Síntesis de Experimentación con Algoritmos de Aprendizaje Automático

1. Entrenamiento basado en Co-ocurrencia y Estructura de Recetas (Random Forest)

- **Dataset utilizado:** Se utilizó un conjunto de datos público proveniente de Food.com (obtenido vía Kaggle), del cual se aislaron y filtraron más de 73,000 recetas exclusivas del rubro de repostería y pastelería.
- **Procesamiento:** Los ingredientes fueron normalizados mediante la unificación de sinónimos culinarios y transformados en una matriz matemática binaria (*One-Hot Encoding*) seleccionando los 100 ingredientes estadísticamente más relevantes.
- **Modelo y Resultados:** Se entrenó un modelo de clasificación supervisada (*Random Forest Classifier*) para predecir la necesidad de un ingrediente estructural (como el huevo) basándose en la presencia del resto de los insumos. El modelo alcanzó una precisión (*Accuracy*) del **79.08%**, demostrando matemáticamente que el algoritmo logra comprender la química culinaria básica (por ejemplo, identificando que la presencia de harina y agentes leudantes exige invariablemente un aglutinante).
- **Conclusión de la iteración:** Aunque el modelo entendió la estructura de la receta, se determinó que la co-ocurrencia estadística no equivale a sustituibilidad. El algoritmo recomendaba ingredientes que suelen acompañar al huevo (como harina o azúcar), evidenciando que un modelo basado puramente en frecuencia no es viable para la recomendación funcional de sustitutos.

2. Entrenamiento basado en Similitud de Propiedades Químicas (K-Nearest Neighbors)

- **Dataset utilizado:** Se estructuró un conjunto de datos de prueba enfocado en las propiedades funcionales de los ingredientes, parametrizando variables como "Poder

Aglutinante", "Nivel de Humedad", "Porcentaje de Grasa" y etiquetas de restricciones alimentarias (ej. "Es Vegano").

- **Modelo y Resultados:** Los datos fueron estandarizados y procesados mediante el algoritmo *K-Nearest Neighbors* (KNN) utilizando distancia euclidiana para encontrar el "vecino más cercano" a nivel químico. Al solicitar una alternativa vegana para el huevo, el modelo recomendó exitosamente sustitutos viables en la repostería real (gel de linaza, plátano molido y puré de manzana) con porcentajes de similitud contextual superiores al 44%.

6.3 Datasets utilizados

- USDA FoodData Central: base de datos publica del gobierno de Estados Unidos con información nutricional detallada de miles de alimentos.
- Literatura culinaria especializada: principalmente el libro BakeWise de Shirley O. Corriher, que explica la química detrás de cada ingrediente en repostería, y The Flavor Bible de Karen Page.

El dataset de prototipo cuenta con 70 ingredientes organizados en 6 grupos: aglutinantes, harinas y almidones, grasas, azúcares y endulzantes, lácteos y alternativas, y leudantes. Cada ingrediente tiene 9 columnas:

Columna	Descripción	Ejemplo (huevo)
nombre	Nombre del ingrediente	huevo
binding_power	Poder aglutinante (0-10)	9
moisture	Aporte de humedad (0-10)	7
fat_content	Contenido de grasa (0-10)	5
sweetness	Nivel de dulzor (0-10)	0
leavening	Capacidad leudante (0-10)	1
is_vegan	Si es apto para dieta vegana	0 (no)
is_gluten_free	Si no contiene gluten	1 (si)
precio_clp	Precio de referencia en pesos chilenos	350

No se requirió una limpieza de datos compleja ya que el dataset fue construido a medida a partir del resultado de la experimentación con el dataset de ingredientes con sus propiedades químicas. Sin embargo, se verificó que todas las propiedades estuvieran en la misma escala (0 a 10) para evitar que valores fuera de rango afectaran el modelo.

6.4 Método de aprendizaje: K-Nearest Neighbors (KNN)

Se eligió el algoritmo K-Nearest Neighbors (KNN) porque el problema de sustituir ingredientes es fundamentalmente un problema de similitud: dado un ingrediente, encontrar cuales otros tienen propiedades culinarias parecidas. Este es exactamente el tipo de problema que KNN resuelve de forma natural.

Un algoritmo de clasificación como Random Forest fue descartado para esta tarea, ¿ya que respondería preguntas del tipo “que ingredientes se suelen llevar con huevos?”, pero no “que ingrediente se parece más al huevo en sus propiedades”. El prototipo inicial contemplaba un intento con Random Forest que demostró no ser el enfoque correcto, ya que el modelo tendía a predecir siempre la clase mayoritaria por el desbalance del dataset.

¿Como funciona el modelo?: Cada ingrediente se representa como un punto en un espacio de 5 dimensiones (una por cada propiedad culinaria). Para encontrar los sustitutos de un ingrediente, el algoritmo busca los K puntos más cercanos en ese espacio, usando la distancia euclidiana como medida de similitud. Los ingredientes que están más cerca tienen propiedades culinarias más similares y por lo tanto son mejores candidatos a ser el reemplazo o sustituto.

Normalización: Antes de calcular distancias, los datos se normalizan con StandardScaler (media 0, desviación estándar 1). Esto es necesario para que ninguna propiedad domine el cálculo solo por tener valores más altos en escala absoluta.

Conversión a porcentaje de similitud: La distancia euclidiana se convierte a un score de similitud entre 0 y 100% usando una función gaussiana: $\text{similitud} = e^{-(d^2 / 2 \cdot \sigma^2)}$. Esto hace que el resultado sea más intuitivo para el usuario final.

6.5 Métricas del prototipo

Al tratarse de un sistema de recomendación sin etiquetas de verdad absoluta (no existe una única respuesta correcta para “que reemplaza al huevo”), la evaluación del modelo se realizó de forma cualitativa y mediante métricas de coherencia:

- Validación cualitativa: se verifico que los sustitutos recomendados fueran coherentes con el conocimiento culinario. Por ejemplo, el modelo coloca al gel de linaza y al gel de chía como los sustitutos más similares al huevo, lo que coincide con la práctica real de la repostería vegana.

- Varianza explicada por PCA: al reducir el espacio de 5 dimensiones a 2 dimensiones para visualización, el análisis de componentes principales (PCA) logra explicar aproximadamente el 75-80% de la varianza del dataset, lo que indica que las 5 propiedades elegidas capturan bien la estructura del espacio culinario.
- Umbral de similitud: solo se guardan pares de sustitutos con un score superior al 10%, filtrando recomendaciones que no tienen sentido culinario real.
- Razón textual automática: el modelo genera automáticamente una explicación legible indicando en que propiedades coinciden el ingrediente original y el sustituto, lo que permite al usuario entender la recomendación.

Es importante aclarar que este prototipo fue desarrollado para demostrar la viabilidad del enfoque y la factibilidad técnica del proyecto. El modelo actual funciona con 70 ingredientes de prueba, lo cual es suficiente para validar el concepto, pero insuficiente para una aplicación en producción.

6.6 Estado actual y limitaciones

A la fecha de este informe, el módulo de IA tiene el siguiente estado:

- El notebook de entrenamiento esta completo y documentado en Google Colab.
- El Endpoints GET /ingredients/{id}/substitutes esta implementado en el Backend y conectado al FrontEnd.
- Los sustitutos actuales en la base de datos son datos semilla de prueba, no resultados del modelo completo.
- El modelo aún no ha sido ejecutado con el dataset completo de 70 ingredientes para generar la sustitución más correcta.

6.7 Trabajo pendiente para el módulo IA

De acuerdo con el plan de trabajo del proyecto, las actividades pendientes para completar este módulo son:

- Ejecutar el notebook KNN en Google Colab con el dataset de 70 ingredientes y generar el archivo CSV de sustitutos con los scores de similitud.
- Importar los resultados del modelo a la base de datos de producción, reemplazando los datos semilla actuales.
- Expandir el catálogo a al menos 100-150 ingredientes para mejorar la cobertura del modelo.

- Incorporar al catálogo los precios reales del mercado chileno, idealmente permitiendo que el usuario los actualice manualmente desde la aplicación.
- Generar las visualizaciones finales del modelo (diagrama PCA de ingredientes, matriz de similitud entre ingredientes clave) para incluirlas en el informe final de título.
- Explorar la incorporación de aprendizaje de refuerzo basado en retroalimentación del usuario (si el usuario acepta o rechaza una sugerencia, el modelo aprende).

7. Plan de Trabajo

El siguiente incremento tiene como objetivo principal completar el módulo de inteligencia artificial.

A continuación se deja un plan de los próximos avances esperados a partir de los siguientes meses:

#	Actividad	Plazo
1	Ejecutar notebook KNN con dataset de 70 ingredientes y exportar CSV	Julio 2026
2	Importar resultados del modelo a PostgreSQL	Julio 2026
3	Agregar los ingredientes adicionales al catálogo de la aplicación	Julio 2026
4	Permitir que el usuario edite los precios de ingredientes desde la app	Agosto 2026
5	Validar cualitativamente los sustitutos con un repostero real	Agosto 2026
6	Generar visualizaciones del modelo para el informe final	Agosto 2026
7	Implementar exportación de cotizaciones en PDF	Septiembre 2026
8	Mejora de detalles visuales de la página	Octubre 2026
9	Fase de QA con usuario	Octubre 2026
10	Finalización de informe de Proyecto de título	Noviembre 2026

8. Referencias

Ingrid Cubas (2023). Los retos que enfrenta la industria de la panadería y pastelería en 2023. THE FOOD TECH.

<https://thefoodtech.com/soluciones-y-tecnologia-alimentaria/los-retos-queenfrenta-la-industria-de-la-panaderia-y-pasteleria-en-2023>

Julia Martins (2025). Scrum: conceptos clave y cómo se aplica en la gestión de proyectos. Asana <https://asana.com/es/resources/what-is-scrum>

PostgreSQL. (2024). PostgreSQL 15 Documentation. <https://www.postgresql.org/docs/15/>

React. (2024). React 18 Documentation. <https://react.dev>

Scikit-learn. (2024). sklearn.neighbors.NearestNeighbors.

<https://scikit-learn.org/stable/modules/neighbors.html>

USDA. (2024). FoodData Central. <https://fdc.nal.usda.gov>

FastAPI. (2024). FastAPI Documentation. <https://fastapi.tiangolo.com>

Vite. (2024). Vite Documentation. <https://vitejs.dev>